

# DSA Final: Dynamic Elevator Simulation

Brandon Esebag

GitHub Repo: <https://github.com/rancorjoy/ElevatorSimulator.git>

# Overview of all Classes (Main Data Structures)

- MainWindow: Displays client window and updates each frame
- AgentController: Controls the behavior of each agent
- Agent: Controls the graphical output of each agent
- ElevatorController: Controls the behavior of each elevator
- ElevatorShaft: Controls the graphical output of each hall segment
- ElevatorCar: Controls the graphical output of each elevator car

# MainWindow Internal Data Structures

- Constant Integers for MaxFloors, MaxShafts and MaxAgents to bound the simulation resources (allowing arrays to be used, cheaper for performance)
- Resulting Arrays:
  - ElevatorController elevatorControllers[MaxShafts]
  - ElevatorCar elevatorCars[MaxShafts]
  - ElevatorShaft elevatorShafts[MaxShafts, MaxFloors]
  - AgentController agentControllers[MaxAgents]
  - Agent agents[MaxAgents]
- These arrays store all elements being simulated, but since the simulation can be shrank we need to deal with all edge cases!

# Add and Remove Shaft(s)

MainWindow:

```
public void AddShaft(int shaft)
{
    if (floorUpDown.Value != null && shaftUpDown.Value != null)
    {
        elevatorControllers[shaft] = new ElevatorController(shaft, this); // Elevator Controller for shaft
        elevatorCars[shaft] = new ElevatorCar(elevatorControllers[shaft], this); // Elevator car for the controller
        for (int f = 0; f < floorUpDown.Value.Value; f++)
        {
            elevatorShafts[shaft, f] = new ElevatorShaft(elevatorControllers[shaft], f, this);
        }
    }
}
1 reference
public void RemoveShaft(int shaft)
{
    if (floorUpDown.Value != null && shaftUpDown.Value != null)
    {
        elevatorControllers[shaft].deactivate(); // Deactivate elevator!
        for (int a = 0; a < MaxAgents; a++)
        {
            if (agentControllers[a] != null)
            {
                agentControllers[a].ValidateShaft(shaftUpDown.Value.Value);
            }
        }
    }
}
```

# Remove Shaft(s) Continued

AgentController:

```
public void ValidateShaft(int shaftCount) // logic for if agent is in a shaft segment that is removed...
{
    if (waitShaft > shaftCount - 1) // if the current shaft position is no longer available
    {
        if (currentState == AgentState.Waiting || currentState == AgentState.Pressing) // if the agent is still waiting or pressing the call button
        {
            waitShaft -= 1; // move to the left one shaft
            xpos -= 1.0f; // move to the left one shaft
        }
        if (currentState == AgentState.Pursuing) // if the agent is pursuing a car
        {
            currentState = AgentState.Waiting; // the agent should not enter the car
            waitShaft -= 1; // move to the left one shaft
            xpos -= 1.0f; // move to the left one shaft
        }
        if (currentState == AgentState.Boarded) // if the agent is in a car
        {
            currentState = AgentState.Leaving; // the agent should leave the simulation
            waitShaft -= 1; // move to the left one shaft
            xpos -= 1.0f; // move to the left one shaft
            ypos = 0.0f; // put agent on bottom floor
            targetFloor = 1; // have the agent exit the simulation
        }
        if (currentState == AgentState.Leaving) // if the agent is on the target floor
        {
            xpos -= 1.0f; // move to the left one shaft
        }
    }
}
```

# Add Floor(s)

MainWindow:

```
public void AddFloor(int floor)
{
    if (floorUpDown.Value != null && shaftUpDown.Value != null)
    {
        for (int s = 0; s < shaftUpDown.Value.Value; s++)
        {
            elevatorShafts[s, floor] = new ElevatorShaft(elevatorControllers[s], floor, this);
        }
    }
}
```

# Removing Floor(s)

In MainWindow:

```
public void RemoveFloor(int floor)
{
    if (floorUpDown.Value != null && shaftUpDown.Value != null)
    {
        upRequests[floorUpDown.Value.Value - 1] = false; // remove requests for floors that are no longer in use!!!
        downRequests[floorUpDown.Value.Value - 1] = false;
        pendingUpRequests[floorUpDown.Value.Value - 1] = false;
        pendingDownRequests[floorUpDown.Value.Value - 1] = false;
        for (int s = 0; s < shaftUpDown.Value; s++) // for all elevators
        {
            elevatorControllers[s].validateFloor(floorUpDown.Value.Value); // validate the floor
        }
        for (int a = 0; a < MaxAgents; a++) // for all agents
        {
            if (agentControllers[a] != null) // if the agent exists (yet)
            {
                if (agentControllers[a].get_active()) // if the agent is still active
                {
                    agentControllers[a].ValidateFloor(floorUpDown.Value.Value); // validate the floor
                }
            }
        }
    }
}
```

# Removing Floor(s) Continued

In ElevatorController(s):

```
// Update this method when a floor is removed! CHANGES STATE!
```

```
1 reference
```

```
public void validateFloor(int floorCount)
```

```
{
```

```
    if (pos > Convert.ToSingle(floorCount - 1)) // if the elevator is higher than the current max floor
```

```
    {
```

```
        // put elevator at first floor and have it open
```

```
        currentState = carStates.opening;
```

```
        Console.WriteLine("Elevator " + shaft + " in State: Opening");
```

```
        delayTimer = 0;
```

```
        pos = 0.0f;
```

```
        floor = 0;
```

```
        doorPos = 0.0f;
```

```
    }
```

```
}
```



# Removing Floor(s) Continued

In Agent(s):

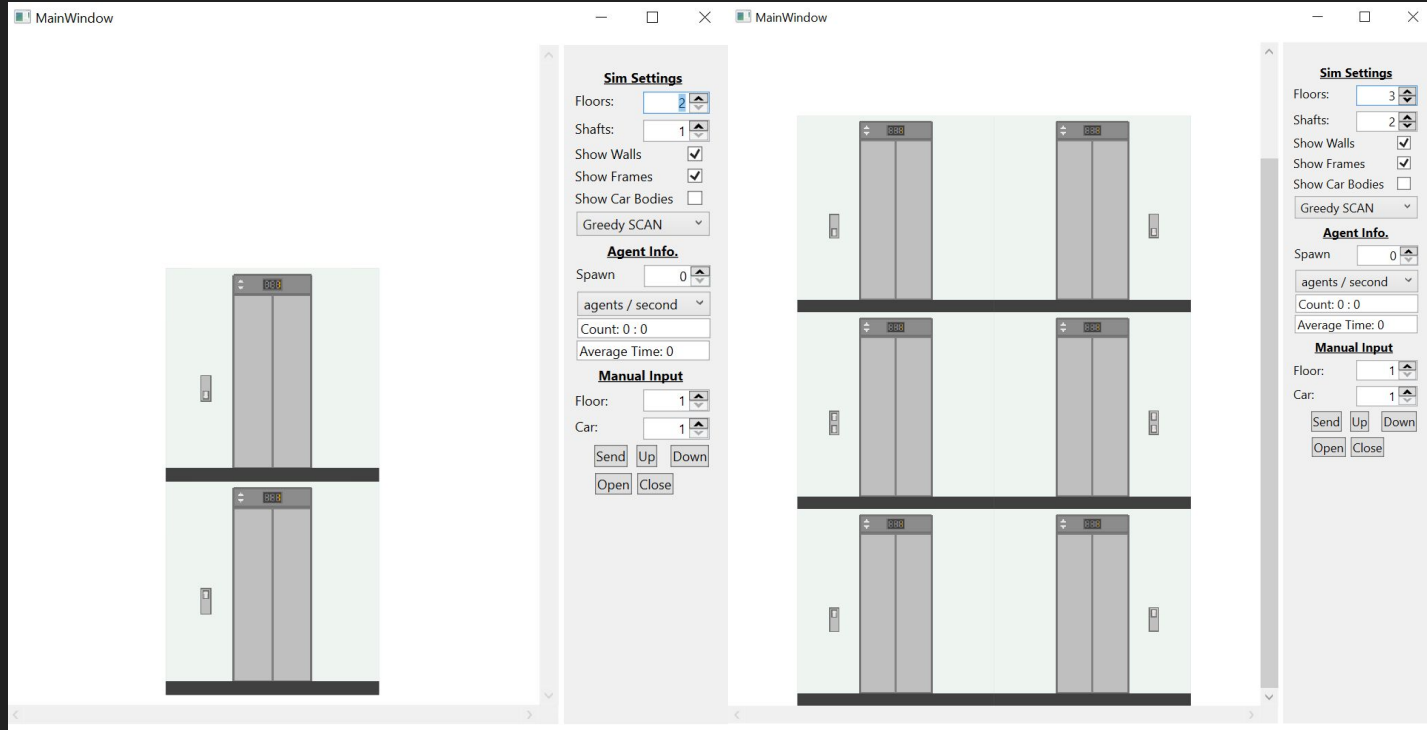
```
public void ValidateFloor(int floorCount) // change target floor if current target is removed
{
    if (targetFloor > floorCount - 1) // if the current target is no longer available...
    {
        if (currentState == AgentState.Waiting || currentState == AgentState.Pressing) // if the agent is still waiting or pressing
        {
            targetFloor = random.Next(floorCount); // choose a new target
            while (initialFloor == targetFloor) // if the initial and target are the same...
            {
                targetFloor = random.Next(floorCount); // regenerate the target floor
            }
        }
    }
    if (currentState == AgentState.Pursuing) // if the agent is pursuing a car
    {
        currentState = AgentState.Waiting; // the agent should not enter the car
        targetFloor = random.Next(floorCount); // choose a new target floor and wait
    }
    if (currentState == AgentState.Boarded) // if the agent is in a car
    {
        targetFloor = 0; // get off on the first floor (where the elevator spawns)
        initialFloor = 1; // ensure the initial floor is not the target floor to prevent bugs (there is always a floor 1)
    }
    if (currentState == AgentState.Leaving) // if the agent is on the target floor
    {
        targetFloor -= 1; // the agent should "target" the previous floor
        ypos -= 1.0f; // the agent is teleported onto the previous floor so that it is not floating
    }
    if (initialFloor > floorCount - 1)
    {
        if (currentState == AgentState.Waiting || currentState == AgentState.Pressing) // if the agent is still waiting or pressing
        {
            ypos -= 1; // move agent to the next floor down
            initialFloor -= 1; // move agent's initialFloor down
        }
    }
}
```

# Changing Dimension Size (MainWindow MainLoop):

```
// Detect Canvas Resizing Event
if (floorUpDown.Value != null && shaftUpDown.Value != null) // If the two dimensions are not null...
{
    if (prev_floors != floorUpDown.Value || prev_shafts != shaftUpDown.Value) // User has changed simulation dimensions!
    {
        ResizeCanvas(floorUpDown.Value.Value, shaftUpDown.Value.Value); // Resize the canvas to fit the new dimensions
    }
}
else // If a dimension value becomes null - reset to default values
{
    floorUpDown.Value = 2;
    shaftUpDown.Value = 1;
}
```

```
// Update simulation dimensions!
if (floorUpDown.Value != null && shaftUpDown.Value != null) // If the two dimensions are not null...
{
    if (prev_floors != floorUpDown.Value.Value)
    {
        if (prev_floors < floorUpDown.Value.Value)
        { // Add floors
            for (int f = prev_floors; f < floorUpDown.Value.Value; f++)
                AddFloor(f);
        }
        else
        { // Remove floors
            for (int f = prev_floors - 1; f >= floorUpDown.Value.Value; f--)
                RemoveFloor(f);
        }
    }
    if (prev_shafts != shaftUpDown.Value.Value)
    {
        if (prev_shafts < shaftUpDown.Value.Value)
        { // Add shafts
            for (int s = prev_shafts; s < shaftUpDown.Value.Value; s++)
                AddShaft(s);
        }
        else
        { // Remove shafts
            for (int s = prev_shafts - 1; s >= shaftUpDown.Value.Value; s--)
                RemoveShaft(s);
        }
    }
}
```

# Changing the Simulation Size (User Interface):



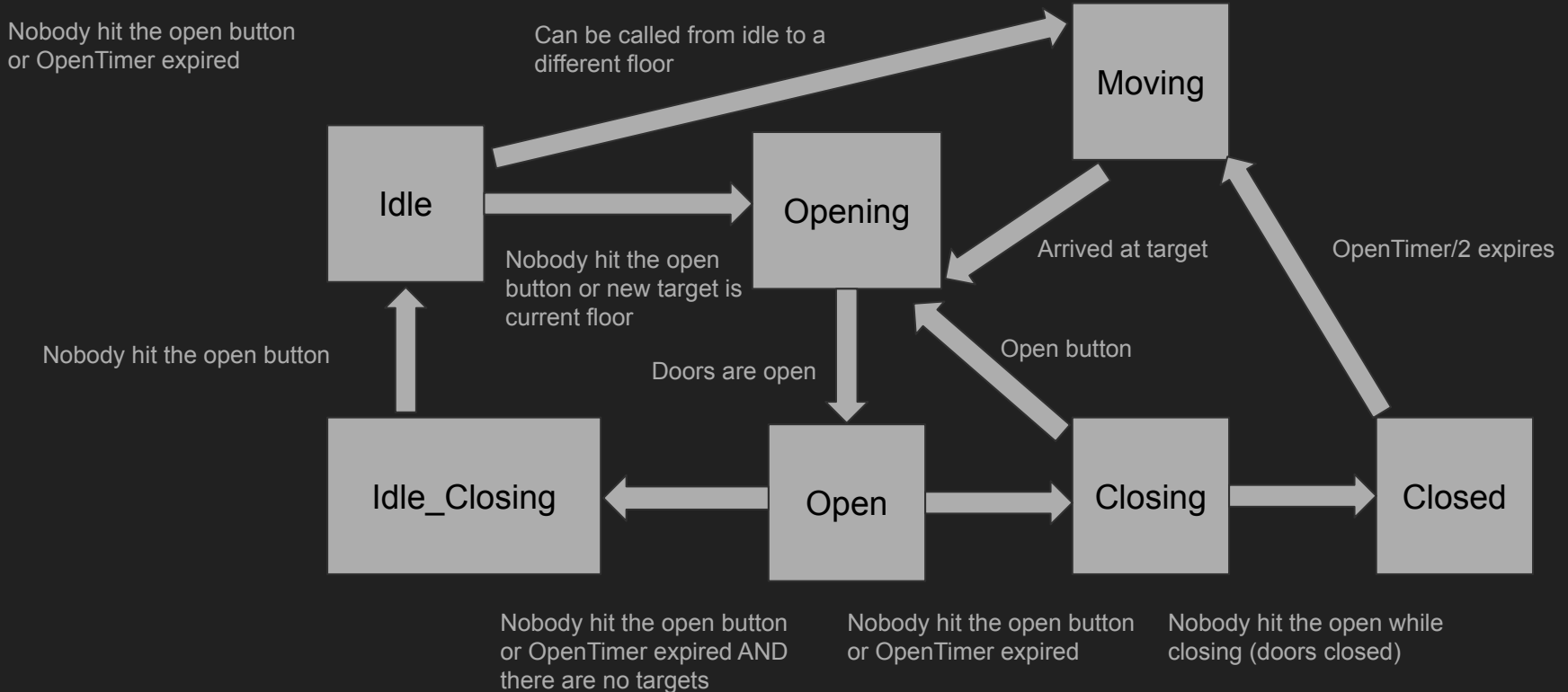
# Elevator Algorithms (SCAN Variants)

Greedy SCAN: Only uses idle elevators if there are no possible moving candidates (minimize idle calling).

Balanced SCAN: Scores each candidate based on distance and chooses the best candidate regardless of idle/moving state.

Aggressive SCAN: Only uses moving elevators if there are no possible idle candidates (maximized idle calling), exact opposite of Greedy SCAN.

# Elevator State Machine (ElevatorController)



# Elevator State Machine Transitions:

```
public void updateState() // should be run each frame!
{
    if (deactivated == false) // if this elevator is still in service
    {
        if (currentState == carStates.idle) // transitions from idle
        {
            delayTimer = 0; // Ensure the timer is reset!
            if (openPressed) // if the open button is pressed (even while in idle)
            {
                currentState = carStates.opening; // start opening the doors
                Console.WriteLine("Elevator " + shaft + " in State: Opening");
                delayTimer = 0;
            }
            else if (floorsPressed.Any(b => b == true)) // if there is a target floor
            {
                nextFloorSet(); // find the next floor to move to!
                currentState = carStates.moving; // if a target floor is given, the elevator starts moving
                Console.WriteLine("Elevator " + shaft + " in State: Moving");
                prevTarget = floor; // set the previous target
            }
            else if (carCapacity != 0) // if the car is somehow idle when it has passengers, it should be opened!
            {
                currentState = carStates.opening; // start opening the doors
                Console.WriteLine("Elevator " + shaft + " in State: Opening");
                delayTimer = 0;
            }
        }

        if (currentState == carStates.idle_closing) ...
        if (currentState == carStates.open) ...
        if (currentState == carStates.opening) ...
        if (currentState == carStates.closing) ...
        if (currentState == carStates.closed) ...
        if (currentState == carStates.moving) ...
    }
}
```

```
if (currentState == carStates.moving) //transitions from moving
{
    if (Math.Abs(pos - Convert.ToSingle(currentTarget)) <= posError) // if the elevator has reached its target
    {
        floor = Convert.ToInt32(MathF.Round(pos)); // ensure the floor updates correctly
        pos = Convert.ToSingle(floor); // round off pos so errors do not accumulate (think of this as the sensor re-tarring)
        currentState = carStates.opening; // start opening the elevator
        Console.WriteLine("Elevator " + shaft + " in State: Opening");
        delayTimer = 0;
        floorsPressed[floor] = false; //remove this floor from the elevator's targets
        if (dir == Direction.up)
        {
            window.clearUp(floor);
        }
        if (dir == Direction.down)
        {
            window.clearDown(floor);
        }
    }
    if (searchFail == true) // if somehow the elevator has no targets (previous checks failed)
    {
        currentState = carStates.idle; // the elevator is idle instead
        Console.WriteLine("Elevator " + shaft + " in State: Idle");
    }
}
```

# Elevator Door and Car Positions:

```
public void updateCarPos()
{
    if (currentState == carStates.moving)
    {
        if (dir == Direction.up) // going up
        {
            if (searchFail == false)
            {
                // Update floor from pos
                floor = (int)Math.Floor(pos); // Round down in this case

                float currentSpeed = 0f;

                // Check which half of the floor we're in
                float floorMidpoint = Convert.ToSingle(floor) + 0.5f;

                if (pos < floorMidpoint) // in lower half of floor (accelerating region)
                {
                    if (prevTarget == floor) // starting from previous floor
                    {
                        currentSpeed = (1.1f * moveSpeed) - (4 * moveSpeed * (pos - Convert.ToSingle(floor) - 0.5f) * (pos - Convert.ToSingle(floor) - 0.5f));
                    }
                    else // not starting from previous floor
                    {
                        currentSpeed = moveSpeed;
                    }
                }
                else // in upper half of floor (decelerating region)
                {
                    if (currentTarget == floor + 1) // stopping at next floor
                    {
                        currentSpeed = moveSpeed - (4 * moveSpeed * (pos - Convert.ToSingle(floor) - 0.5f) * (pos - Convert.ToSingle(floor) - 0.5f));
                    }
                    else // not stopping at next floor
                    {
                        currentSpeed = moveSpeed;
                    }
                }
            }
        }
    }
}
```

```
        pos += currentSpeed / framerate;
    }
}

if (dir == Direction.down) // going down
{
    if (searchFail == false)
    {
        // Update floor from pos
        floor = (int)Math.Ceiling(pos); // Round up in this case

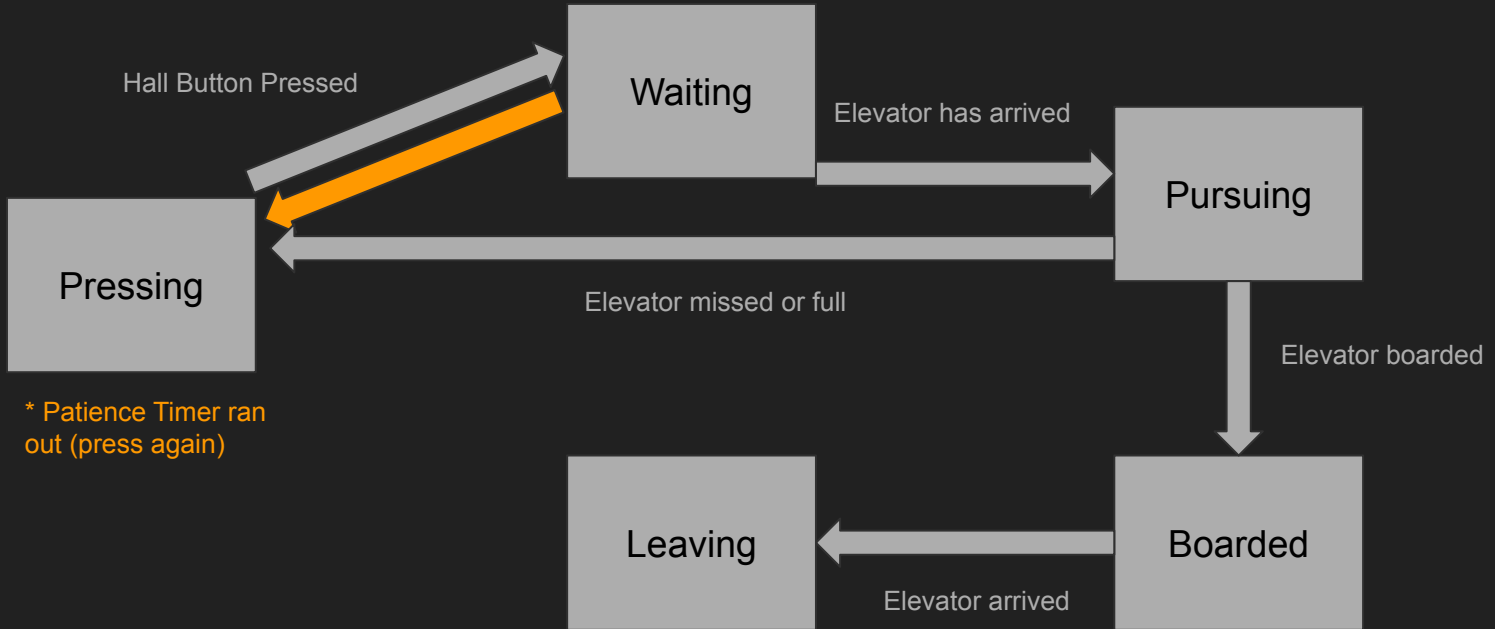
        float currentSpeed = 0f;

        // Check which half of the floor we're in
        float floorMidpoint = Convert.ToSingle(floor) - 0.5f;

        if (pos > floorMidpoint) // in upper half of floor (accelerating region)
        {
            if (prevTarget == floor) // starting from previous floor
            {
                currentSpeed = (1.1f * moveSpeed) - (4 * moveSpeed * (Convert.ToSingle(floor) - pos - 0.5f) * (Convert.ToSingle(floor) - pos - 0.5f));
            }
            else // not starting from previous floor
            {
                currentSpeed = moveSpeed;
            }
        }
        else // in lower half of floor (decelerating region)
        {
            if (currentTarget == floor - 1) // stopping at next floor
            {
                currentSpeed = moveSpeed - (4 * moveSpeed * (Convert.ToSingle(floor) - pos - 0.5f) * (Convert.ToSingle(floor) - pos - 0.5f));
            }
            else // not stopping at next floor
            {
                currentSpeed = moveSpeed;
            }
        }
    }
}
```

There is more... this is the main section

# Agent State Machine (AgentController)





# Agent State Transitions

```
public void UpdateState(int shaftCount, int framerate)
{
    if (currentState == AgentState.Pressing) ...
    else if (currentState == AgentState.Waiting) ...
    else if (currentState == AgentState.Pursuing)
    {
        if (MathF.Abs(xpos - waitShaft) < 0.1) // if the car is here and agent is close enough to board...
        {
            if (controllers[waitShaft].board())
            {
                currentState = AgentState.Boarded;
                controllers[waitShaft].hitFloor(targetFloor);
                Boarded = true;
            }
        }
        else if (controllers[waitShaft].get_doorPos() == 0.0f) // if waited elevator is/was here not open anymore
        {
            currentState = AgentState.Pressing;
            if(xpos < (shaftCount - 0.5)/2) // agent closer to the left
            {
                waitShaft = 0; // reset the wait shaft
            }
            else // agent closer to the right
            {
                waitShaft = shaftCount - 1; // reset the wait shaft
            }
        }
    }
    else if (currentState == AgentState.Boarded) ...
    else if (currentState == AgentState.Leaving) ...
}
```

```
public void UpdateState(int shaftCount, int framerate)
{
    if (currentState == AgentState.Pressing) ...
    else if (currentState == AgentState.Waiting)
    {
        if (controllers[waitShaft].get_floor() == initialFloor && controllers[waitShaft].get_doorPos() == 1.0f) // if waited elevator is here and open
        {
            currentState = AgentState.Pursuing;
        }
        if(patientceTimer / framerate >= patience) // if agent has waited too long...
        {
            patientceTimer = 0; // reset patience timer
            currentState = AgentState.Pursuing; // press the button again!
        }
    }
    else if (currentState == AgentState.Pursuing) ...
    else if (currentState == AgentState.Boarded) ...
    else if (currentState == AgentState.Leaving) ...
}
```

# Agent Movement

```
public void UpdatePos(int shaftCount, int framerate)
```

```
{
    if (Active) // if this agent is still active
    {
        // Update the timer!
        lifeTimer += 1;
        patientceTimer += 1;

        // Get button targets
        float leftButtonPos = -0.4f;
        float rightButtonPos = (shaftCount - 1) + 0.4f;
        float buttonRange = 0.05f;

        // Determine if agent is near button(s) or not
        if (shaftCount > 1)
        if (Mathf.Abs(xpos - leftButtonPos) < buttonRange)

        // Set (arbitrary) error tollerance)
        float errorTol = 0.02f;

        // Get current speed
        float currentSpeed = speed / Convert.ToSingle framerate);

        // Get wait target
        float waitTarget = Convert.ToSingle(waitShaft); // this would have user wait right on shaft center like a prick (possible)
        if (initialLR == false || shaftCount == 1)
        else if (initialLR)

        // Get board target
        float boardPos = Convert.ToSingle(waitShaft); // this would have the agent stand in the center of the car (possible)
        if (carPos > 0.5f)
        else if (carPos < 0.5f)

        // STATE LOGIC
        if (currentState == AgentState.Pressing)
        else if (currentState == AgentState.Waiting)
        else if (currentState == AgentState.Pursuing)
        else if (currentState == AgentState.Boarded)
        else if (currentState == AgentState.Leaving)
    }
}
```

```
// STATE LOGIC
```

```
if (currentState == AgentState.Pressing)
else if (currentState == AgentState.Waiting)
else if (currentState == AgentState.Pursuing)
{
    if (xpos > waitShaft && Mathf.Abs(xpos - waitShaft) > errorTol)
    if (xpos < waitShaft && Mathf.Abs(xpos - waitShaft) > errorTol)
    if (Mathf.Abs(xpos - waitShaft) < 0.15 && controllers[waitShaft].canBoard()) // if the agent is close enough and elevator has room
    {
        controllers[waitShaft].openPress(); // open the elevator
    }
}
else if (currentState == AgentState.Boarded)
{
    Boarded = true;
    if (controllers[waitShaft].get_floor() == targetFloor)
    else
    if (xpos > boardPos && Mathf.Abs(xpos - boardPos) > errorTol)
    if (xpos < boardPos && Mathf.Abs(xpos - boardPos) > errorTol)
    ypos = controllers[waitShaft].get_pos(); // move up/down with the elevator!
}
else if (currentState == AgentState.Leaving)
```